Day 1 Appendix — Advanced Techniques & Tools

> 🎯 **Goal**: Deep-dive into the training tricks, architecture innovations, and inference optimizations that make modern LLMs work at scale.

> ⏱️ **Time**: Optional deep-dive — 2–3h (reference material, revisit as needed)

> 📦 **Deliverable**: A mental model of how every major LLM optimization works, why it exists, and when to use it

---

Table of Contents

1. Training Essentials
   - Cross-Entropy Loss
   - Adam / AdamW Optimizer
   - Learning Rate Schedule
   - Mixed-Precision Training (BF16/FP16)
2. Modern Architecture Tricks
   - Mixture of Experts (MoE)
   - Grouped-Query Attention (GQA)
   - Flash Attention
   - RMSNorm vs LayerNorm
3. Fine-Tuning Methods
   - LoRA (Low-Rank Adaptation)
   - QLoRA
   - Decision Table: When to Use What
4. Inference Speedups
   - Quantization (INT8 / INT4)
   - Speculative Decoding
   - PagedAttention / vLLM
   - KV-Cache Compression
5. Evaluation & Scaling
   - Perplexity
   - Key Benchmarks
   - Chinchilla Scaling Law

- Inference-Time Compute Scaling

Training Essentials

Cross-Entropy Loss

Intuition: "How surprised is the model by the correct answer?"

When a model sees the sentence "The cat sat on the ___", it produces a probability for every word in its vocabulary. Cross-entropy loss measures how much probability it assigned to the **right** answer. If the model is confident and correct → low loss. If the model is confident and **wrong** → very high loss.

```
The formula:

  L = -log(P(correct_token))

Breaking it down:

  P(correct_token)  →  The probability the model assigned to the right answer.
                       This is a number between 0 and 1.

  -log(...)         →  Converts probability into "surprise."
                       High probability → low surprise → low loss.
                       Low probability  → high surprise → high loss.

  The minus sign (-) →  log of a number < 1 is negative, so we negate
                        to get a positive loss value.
```

Example — seeing the numbers:

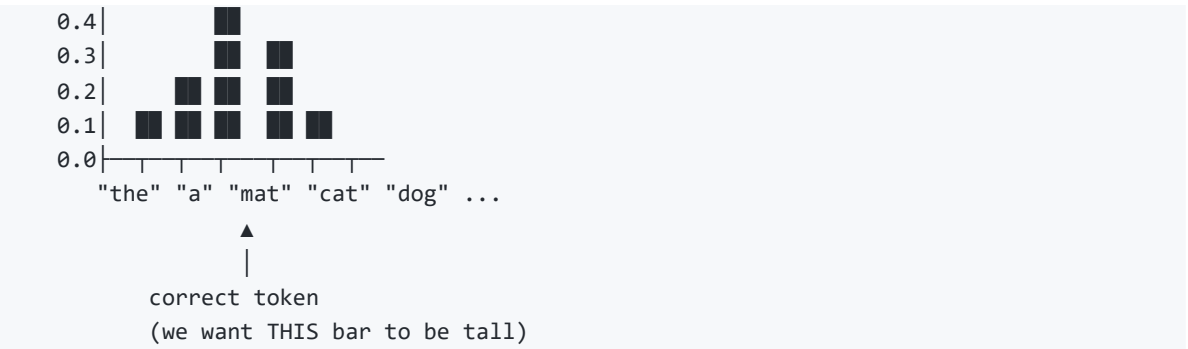
```
Suppose the correct next token is "mat":

  Model A (good):    P("mat") = 0.95  →  L = -log(0.95) = 0.05   ✓ Low loss!
  Model B (unsure):  P("mat") = 0.30  →  L = -log(0.30) = 1.20   ⚠ Medium loss
  Model C (wrong):   P("mat") = 0.01  →  L = -log(0.01) = 4.60   ✗ Terrible!

Notice: the loss doesn't just go up linearly — it EXPLODES when the model
is confidently wrong. That's exactly what we want. A model that says
"I'm 99% sure it's 'banana'" when the answer is "mat" gets punished HARD.
```

```
Probability distribution over vocabulary:

  P  |
  0.6|       ██
  0.5|       ██
```



Why cross-entropy and not just "% correct"? Two reasons:

1. **Differentiable** — "% correct" is either 0 or 1, so gradients can't flow through it. Cross-entropy gives smooth gradients for optimization.
2. **Penalizes confident mistakes** — A model that puts 50% on the right answer and 50% on one wrong answer gets a much lower loss than a model that puts 1% on the right answer and 99% on a wrong answer. "% correct" would score both as "correct" with the same 1.

Adam / AdamW Optimizer

Analogy: Imagine a ball rolling downhill trying to find the lowest valley (the best model weights).

- **Plain SGD** = a ball on ice. It slides around wildly, overshoots valleys, gets stuck oscillating.
- **SGD + Momentum** = a ball with some weight. It builds up speed in consistent directions and pushes through small bumps.
- **Adam** = a ball with friction AND adaptive wheel size. It moves faster on gentle, consistent slopes and automatically slows down on steep, noisy terrain.

What Adam actually does — two key ideas:

```
1. MOMENTUM (m) — "Where have I been heading?"
   Keeps a running average of recent gradients.
   If gradients consistently point left, momentum builds up → bigger step left.
   If gradients flip-flop randomly, momentum stays small → cautious steps.

2. ADAPTIVE LEARNING RATE (v) — "How bumpy is this terrain?"
   Keeps a running average of squared gradients.
   For parameters with large, volatile gradients → v is big → step shrinks.
   For parameters with small, consistent gradients → v is small → step grows.
```

The update formula (explained):

```
θ = θ − lr × m̂ / (√v̂ + ε)
```

```
θ        →  The model parameter we're updating (one weight)
lr       →  Learning rate — the base step size (typically 1e-4 to 3e-4 for LLMs)
m̂        →  Smoothed gradient direction (momentum term, bias-corrected)
√v̂       →  Smoothed gradient magnitude (how bumpy this parameter's landscape is)
ε        →  Tiny number (~1e-8) to prevent division by zero

In plain English: "Take a step proportional to the smoothed gradient,
but shrink it if this parameter has been volatile."
```

AdamW — the modern variant:

```
Standard Adam:   θ = θ − lr × (m̂/(√v̂+ε) + λ×θ)    ← weight decay mixed in
AdamW:           θ = θ − lr × m̂/(√v̂+ε) − lr×λ×θ    ← weight decay SEPARATED

Why it matters: In standard Adam, the adaptive learning rate accidentally
scales the weight decay too. AdamW fixes this by decoupling them.

Result: better generalization, more predictable regularization.
AdamW is the default optimizer for virtually all modern LLM training
(GPT-5, Claude Opus 4.6, LLaMA 4, Gemini 3).
```

Key hyperparameters:

- `lr`: 1e-4 to 3e-4 (typical for LLM pre-training)
- `β₁`: 0.9 (momentum decay — how much to remember past gradients)
- `β₂`: 0.999 (adaptive LR decay — how much to remember past magnitudes)
- `weight_decay (λ)`: 0.01 to 0.1 (prevents weights from growing too large)

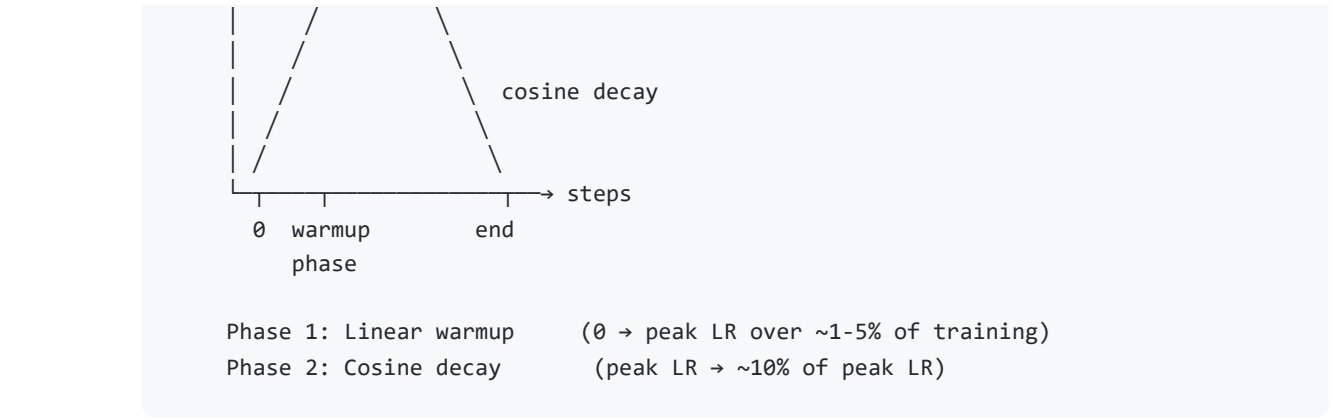
Learning Rate Schedule

Analogy: Warming up a car engine on a cold morning.

1. **Warmup** — Start the engine gently. If you floor the gas immediately (high LR from step 0), the engine stalls (random initial weights + high LR = chaotic updates the model never recovers from).
2. **Peak** — Cruise at full speed. The model has found the general landscape and can take confident steps.
3. **Decay** — Ease into the destination. As you approach a good solution, slow down so you don't overshoot it.

```
Learning Rate over Training Steps:

    LR
    |       /————\
    |      /      \
```




```
Phase 1: Linear warmup     (0 → peak LR over ~1-5% of training)
Phase 2: Cosine decay       (peak LR → ~10% of peak LR)
```

Why warmup is critical:

```
Without warmup:
  Step 0: Weights are random. Gradients are HUGE and noisy.
  High LR × huge gradient = MASSIVE weight update.
  The model jumps to a terrible region of weight space.
  It never recovers → training diverges (loss goes to infinity).

With warmup:
  Step 0: LR is tiny (e.g., 1e-7). Even with noisy gradients,
  the updates are small. The model gently finds its footing.
  Gradually ramp up to full LR (e.g., 3e-4) by step 2000.
  By then, gradients are stable → full speed ahead.
```

Typical values for LLM training:

- Warmup: 1–5% of total training steps (e.g., 2000 steps out of 100K)
- Peak LR: 3e-4 (common for models up to ~7B)
- Final LR: 10% of peak (3e-5) — some trainers go to 0
- Schedule: Cosine decay is the most popular (smooth, no sudden jumps)

Mixed-Precision Training (BF16/FP16)

Analogy: Using pencil sketches (BF16) for draft work and ink (FP32) for the final drawing. Most of the computational work doesn't need full precision — you can sketch faster and still get the same picture.

What the precision formats look like:

```
┌──────────────────────────────────────────────────────────────────┐
│                    NUMBER FORMATS COMPARED                       │
│                                                                  │
│  FP32 (32 bits):  [1 sign][8 exponent][23 mantissa]              │
│     → Full precision. The "source of truth."                     │
│     → Range: ±3.4 × 10³⁸, ~7 decimal digits precision            │
```

```
┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
│  FP16 (16 bits):  [1 sign][5 exponent][10 mantissa]              │
│     → Half the bits → half the memory, ~2× faster matmuls        │
│     → Range: ±65504 only! Can overflow during training.          │
│     → Needs "loss scaling" to avoid underflow in gradients.      │
│                                                                  │
│  BF16 (16 bits):  [1 sign][8 exponent][7 mantissa]               │
│     → Same exponent range as FP32 (no overflow!)                 │
│     → Less decimal precision, but that's fine for neural nets.   │
│     → No loss scaling needed. Simpler, more stable.              │
│     → Preferred by: GPT-5, Claude Opus 4.6, LLaMA 4, Gemini 3    │
└──────────────────────────────────────────────────────────────────┘
```

What runs in which precision:

```
┌──────────────────────────────────┬──────────────┐
│ Operation                        │ Precision    │
├──────────────────────────────────┼──────────────┤
│ Master copy of weights           │ FP32         │
│ Forward pass (matrix multiplies) │ BF16         │
│ Backward pass (gradient compute) │ BF16         │
│ Weight update (optimizer step)   │ FP32         │
│ Loss computation                 │ FP32         │
└──────────────────────────────────┴──────────────┘

Why keep master weights in FP32:
  Small gradient updates (e.g., 0.00001) would vanish in BF16
  (it only has ~3 decimal digits of precision).
  FP32 accumulates these tiny updates correctly over time.
```

Performance gains:

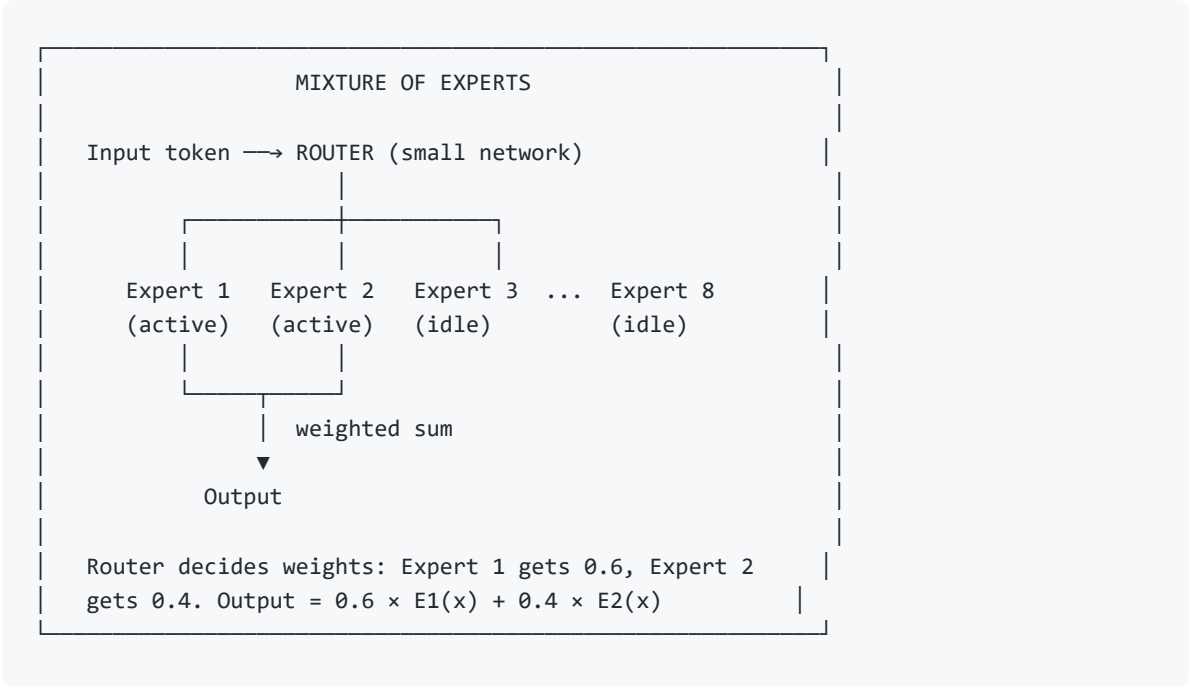
- **Speed**: ~2× faster matrix multiplications on modern GPUs (A100, H100)
- **Memory**: ~50% reduction in activation memory
- **BF16 vs FP16**: BF16 is strictly preferred on modern hardware. Same speed, no overflow risk, no loss scaling hassle.

Modern Architecture Tricks

Mixture of Experts (MoE)

Analogy: Imagine a large company with 8 specialist departments — tax, legal, engineering, marketing, HR, sales, R&D, and support. When a request comes in, the **receptionist** (router) reads it and sends it to only the **2 most relevant departments**. The other 6 departments don't even see it.

Why it matters: A model with 8 experts has 8× the total parameters (knowledge capacity) but only activates 2 experts per token (compute cost). You get the **knowledge** of a massive model with the **speed** of a much smaller one.



Real-world MoE models:

Model	Total Params	Active/Token	Experts
GPT-4 (rumored)	~1.8T	~220B	8 experts
GPT-5 / 5.1 / 5.2	Likely MoE	Likely MoE	Undisclosed
Mixtral 8×7B	47B	~13B	8 experts
LLaMA 4 Maverick	~400B	Subset	17 experts
DeepSeek-V2	236B	~21B	160 experts

Key insight: MoE models are **big in storage** but **fast in inference**. A 47B-parameter Mixtral model runs at roughly the speed of a 13B dense model — because only 13B parameters are active per token.

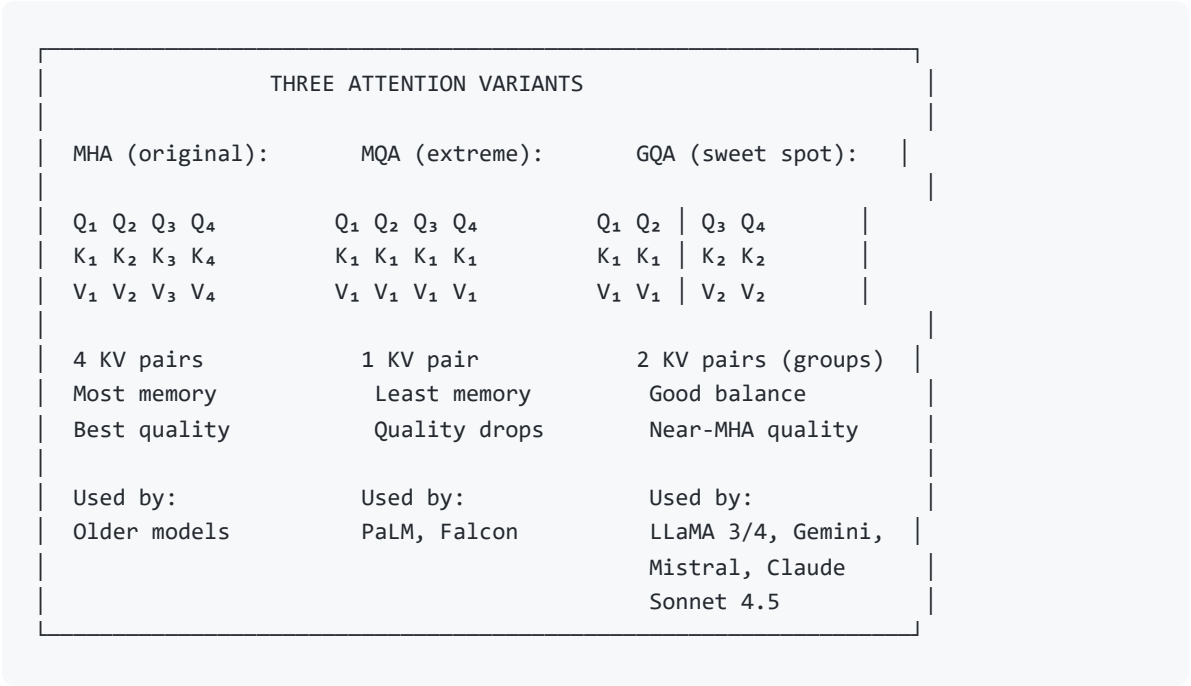
The router balancing problem: If the router keeps sending all tokens to the same 2 experts, the other 6 learn nothing. Training adds an **auxiliary load-balancing loss** that penalizes uneven expert usage, ensuring all experts develop useful specializations.

Grouped-Query Attention (GQA)

The problem: In standard Multi-Head Attention (MHA), every head has its own Query (Q), Key (K), and Value (V) projections. During inference, the K and V values for every past token are

stored in the **KV-cache** so they don't need to be recomputed. With 64 attention heads, that means 64 separate K and 64 separate V matrices per token — consuming enormous GPU memory.

The solution: Share K and V across **groups** of query heads. Instead of 64 unique KV pairs, you might have only 8. Each group of 8 query heads shares one K and one V.



Memory savings: With 64 query heads and 8 KV groups (GQA-8), you reduce the KV-cache by **8×**. For a 70B model serving thousands of concurrent users, this is the difference between needing 8 GPUs and needing 2.

When to use:

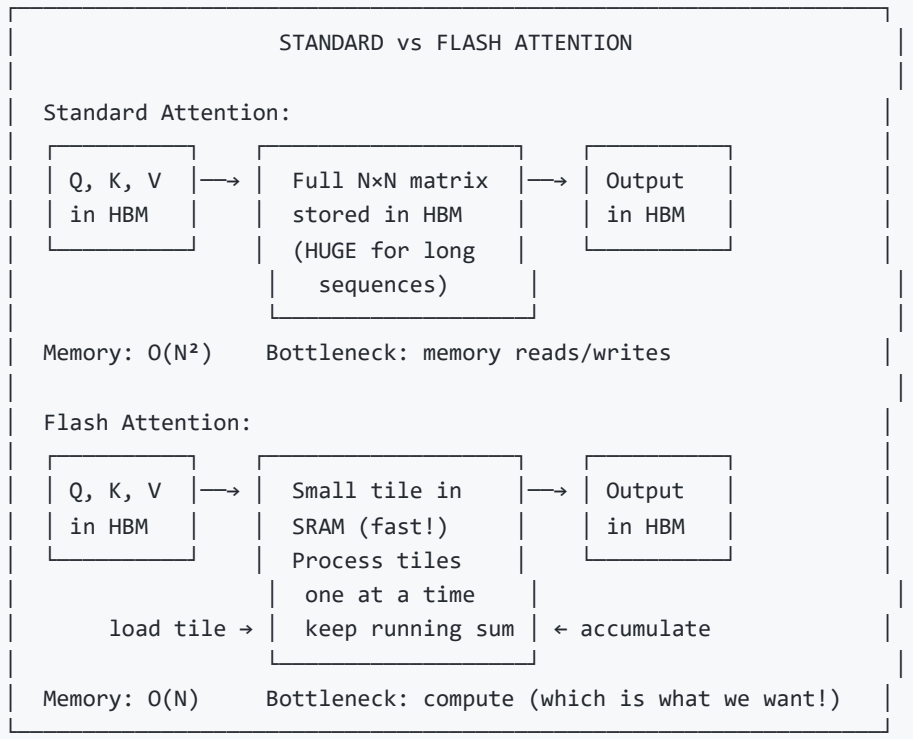
- MHA: Research, small models, when quality is the only priority
- MQA: Extreme latency requirements (e.g., real-time gaming NPCs)
- GQA: Production LLMs (the standard for virtually all modern models)

Flash Attention

The problem: Standard attention computes the full N×N attention matrix and stores it in GPU HBM (High-Bandwidth Memory). For a sequence of 100K tokens, that matrix has 10 **billion** entries. The GPU spends most of its time moving data between memory levels — not actually computing.

Analogy: Imagine processing a massive spreadsheet. The standard way: load the ENTIRE spreadsheet into RAM, compute every cell, then write results back. Flash Attention: process

small tiles of the spreadsheet one at a time, keeping running totals in fast on-chip SRAM. Same exact answer, but you never load the whole spreadsheet at once.



Key insight: Flash Attention is **NOT an approximation**. It computes the **exact same result** as standard attention, just with smarter memory access patterns. The math is identical — it's purely an engineering optimization.

Why it matters:

- **2–4× faster** attention computation
- **O(N) memory** instead of O(N^2) — enables context windows of 100K+ tokens
- Used in: virtually every modern LLM training and inference stack
- Flash Attention 2/3: further optimizations for even better hardware utilization

---

RMSNorm vs LayerNorm

Both are normalization techniques that stabilize training by keeping activations in a reasonable range. Think of them as "volume control" for neural network signals.

LayerNorm (original, used in GPT-2/3, BERT):

```
norm(x) = γ × (x − μ) / σ + β

Step 1: Subtract the mean (μ) — center the values around zero
```

```
Step 2: Divide by standard deviation (σ) — normalize the spread
Step 3: Scale by γ (learned) and shift by β (learned) — let the
        model decide the final range
```

RMSNorm (modern, used in LLaMA 3/4, Mistral, GPT-NeoX, Gemini 3):

```
norm(x) = γ × x / RMS(x)

Where RMS(x) = √(mean(x²))

Key difference: SKIPS the mean subtraction step entirely.
Just divides by the root-mean-square of the values.
No β (shift) parameter either — simpler.
```

Why RMSNorm wins:

- **~10–15% faster**: Skipping mean computation and the shift parameter saves time per layer, and transformers have many layers.
- **Works equally well in practice**: Empirically, the mean subtraction doesn't help much for transformer language models.
- **Simpler**: Fewer parameters, fewer operations, easier to implement efficiently on hardware.

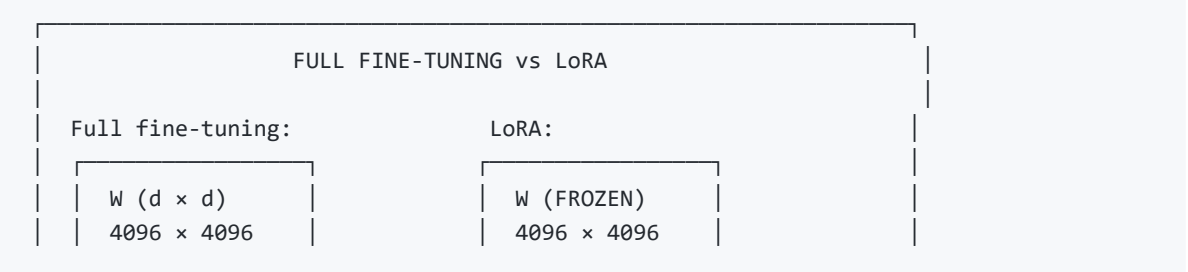
When to use: If you're training a new model, use RMSNorm. LayerNorm is only relevant for understanding older architectures.

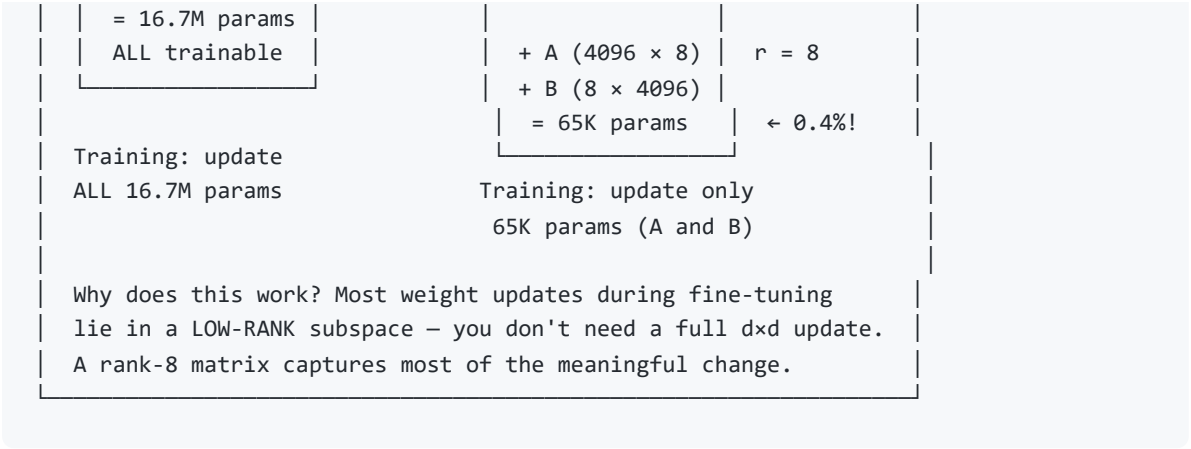
Fine-Tuning Methods

LoRA (Low-Rank Adaptation)

Analogy: Instead of rewriting an entire textbook for a new course (full fine-tuning, updating all weights), you add **sticky notes** with corrections and additions to specific pages. The original textbook stays completely unchanged. When you read a page, you see the original content + your sticky notes combined.

What it does: Freezes the original weight matrix W (the "textbook"). Adds a tiny trainable decomposition on top: W' = W + A × B, where A and B are much smaller matrices.





The math (simplified):

```
Original:   output = W × input         (frozen, not updated)
LoRA adds:  output = W × input + A × B × input

W is (d × d)  = 4096 × 4096 = 16,777,216 params (FROZEN)
A is (d × r)  = 4096 × 8    =      32,768 params (trainable)
B is (r × d)  = 8 × 4096    =      32,768 params (trainable)

Total trainable: 65,536 params = 0.39% of W

r = "rank" — controls capacity. Think of it as the number of
"sticky note dimensions." Higher rank = more capacity = more params.
```

Typical configuration:

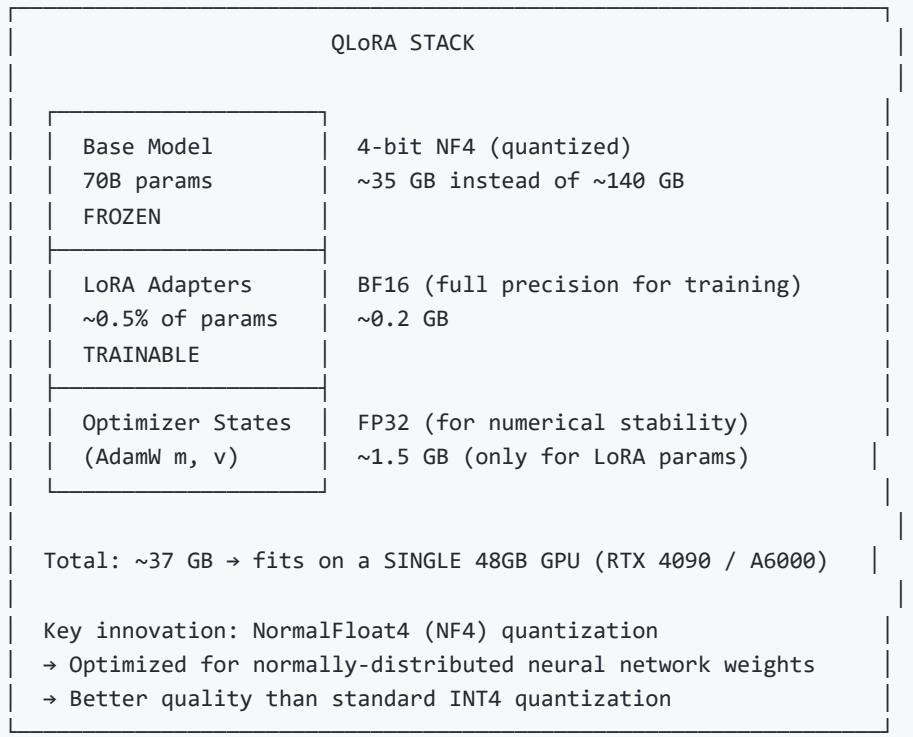
- **Rank**: r = 8 to r = 64. Start with r = 16, increase if quality is insufficient.
- **Which layers**: Usually attention projection matrices (Q, V). For more capacity, apply to all linear layers (Q, K, V, O, and FFN).
- **Alpha**: A scaling factor (usually α = 2×r). Controls how much the LoRA update influences the output.

When to use LoRA: Style changes, format adaptation, moderate domain shift. NOT for teaching the model fundamentally new knowledge (use full fine-tuning for that).

QLoRA

The problem: Even with LoRA, you still need to load the **full base model** in memory for the forward pass. A 70B parameter model in FP16 requires ~140 GB of GPU memory — that's 2× A100 80GB GPUs just to load the weights.

The solution: Quantize the base model to **4-bit** (NF4) for storage and forward pass. Keep the LoRA adapter weights in BF16 for training precision.



When to use QLoRA: When you want LoRA fine-tuning but don't have access to multi-GPU setups. Quality is slightly below standard LoRA but the accessibility trade-off is excellent.

Decision Table: When to Use What

	Full Fine-Tune	LoRA	QLoRA	Prompt Engineering
Training data	10K–1M+	1K–100K	1K–100K	0 (just prompts)
GPU memory	8×A100	1–2×A100	1×RTX4090	None
Training time	Days–weeks	Hours	Hours	Minutes
Quality ceiling	Best	Near-best	Good	Depends on task
Model changes	All weights	Adapters	Adapters	No weights
Cost	$$$$$	$$$	$$	$
When to use	New domain, specialized knowledge	Style / format changes	Budget-limited hardware	Try this FIRST! Often enough.

```
Rule of thumb:
  1. Start with prompt engineering (free, instant)
  2. If not enough, try LoRA (hours, one GPU)
  3. If tight on hardware, use QLoRA (same quality, less memory)
  4. Full fine-tune only when you need deep domain knowledge changes
```

Inference Speedups

Quantization (INT8 / INT4)

Analogy: JPEG compression for model weights. You lose a tiny bit of precision, but save massive amounts of memory and run faster. Just like a JPEG photo looks 99% as good as the raw file at 10% the size.

How it works: Convert FP16 weights (16 bits per number) to INT8 (8 bits) or INT4 (4 bits). Fewer bits per weight = less memory = faster memory transfers = faster inference.

```
┌──────────┬────────────┬──────────┬──────────────────┬─────────────────┐
│ Precision│ Bits/param │ Memory   │ Speed            │ Quality Loss    │
├──────────┼────────────┼──────────┼──────────────────┼─────────────────┤
│ FP16     │ 16         │ 1× (ref) │ 1× (ref)         │ None            │
│ INT8     │ 8          │ 0.5×     │ ~1.5× faster     │ Minimal         │
│ INT4     │ 4          │ 0.25×    │ ~2× faster       │ Small           │
│ 2-bit    │ 2          │ 0.125×   │ ~3× faster       │ Noticeable      │
└──────────┴────────────┴──────────┴──────────────────┴─────────────────┘

70B model at different precisions:
  FP16: ~140 GB → needs 2× A100 80GB
  INT8:  ~70 GB → fits 1× A100 80GB
  INT4:  ~35 GB → fits 1× RTX 4090 (48GB)
  2-bit: ~18 GB → fits 1× RTX 4080 (16GB)... but quality degrades
```

Popular quantization methods:

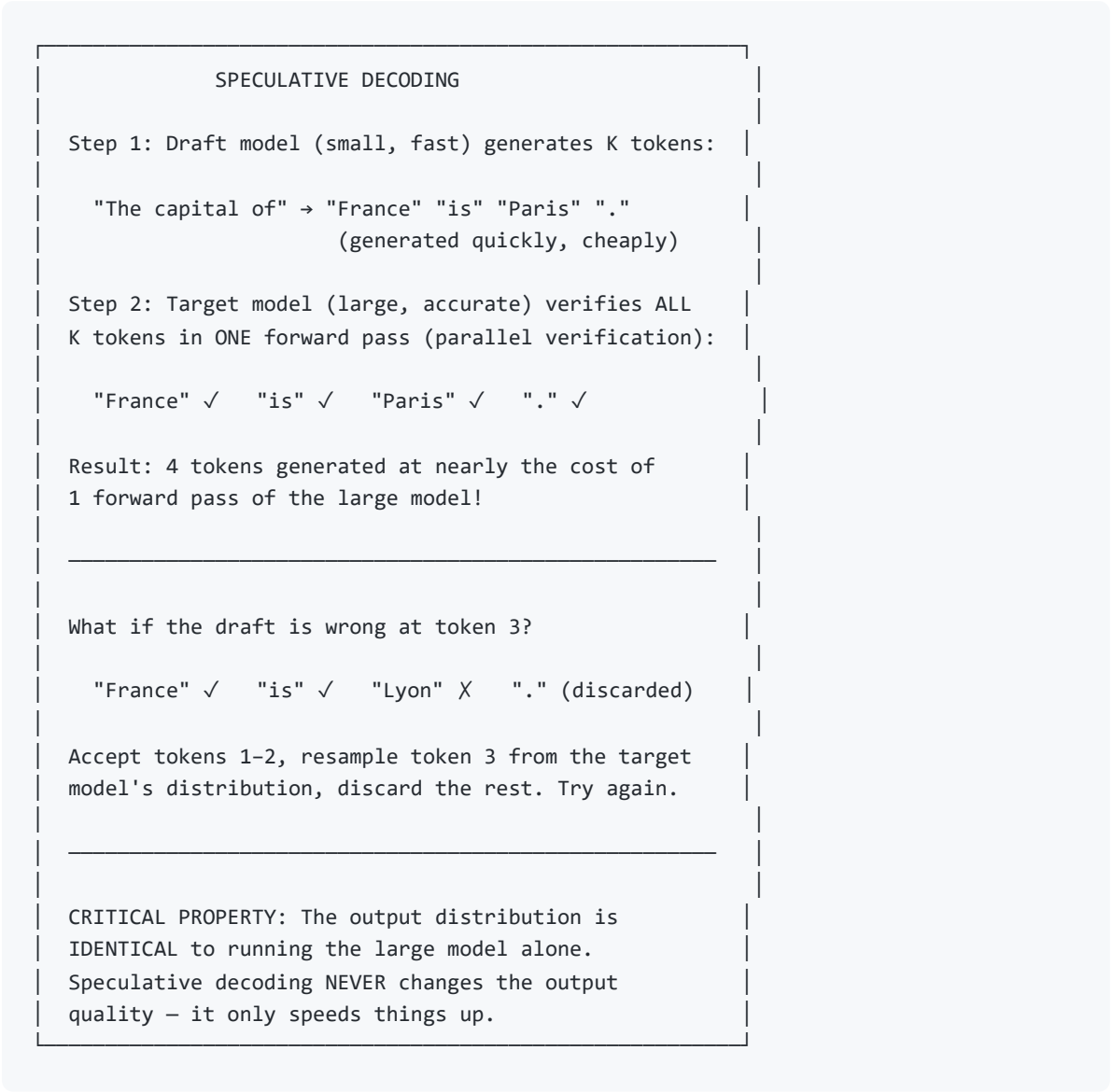
```
┌──────────────────┬──────────────────────────────────────────────────┐
│ Method           │ How It Works                                     │
├──────────────────┼──────────────────────────────────────────────────┤
│ GPTQ             │ Post-training, layer-by-layer. Fast to apply.    │
│                  │ Good quality. Very popular for local models.     │
├──────────────────┼──────────────────────────────────────────────────┤
│ AWQ              │ Activation-aware. Keeps important weights at     │
│                  │ higher precision. Better quality than GPTQ.      │
├──────────────────┼──────────────────────────────────────────────────┤
│ bitsandbytes     │ Easy integration with HuggingFace. Just add      │
│                  │ load_in_4bit=True. Great for quick experiments.  │
├──────────────────┼──────────────────────────────────────────────────┤
│ GGUF (llama.cpp) │ CPU-friendly format. Run models on laptops       │
│                  │ without a GPU. Various quant levels (Q4, Q5).    │
└──────────────────┴──────────────────────────────────────────────────┘
```

When to use: Almost always in production. INT8 has negligible quality loss. INT4 is the sweet spot for running large models on consumer hardware.

Speculative Decoding

Analogy: An intern writes a draft email (fast but imperfect). The senior manager reviews the draft and accepts or revises each sentence (slow but accurate). Net result: faster than the senior writing from scratch, because reviewing is faster than writing.

How it works:



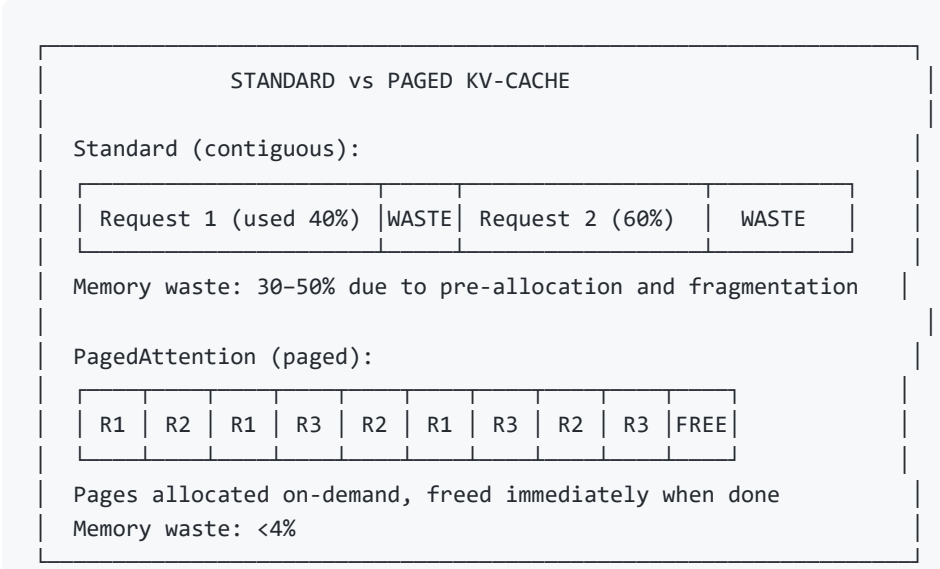
Speedup: Typically **2–3×** depending on the draft model's acceptance rate. If the draft model agrees with the target model 80% of the time, most draft tokens get accepted and you skip expensive sequential generation.

When to use: Production inference of large models (GPT-5, Claude Opus 4.6) where latency matters. The draft model is usually a smaller variant from the same family (e.g., a 1B draft for a 70B target).

PagedAttention / vLLM

The problem: During inference, every active request maintains a **KV-cache** — the stored keys and values for all past tokens. In standard serving, memory for each request is allocated in one big contiguous block. With many concurrent users, GPU memory becomes fragmented — like a parking lot where every car has a reserved row of 20 spots, but most cars only use 5.

The solution: Borrowed from operating systems. Just like virtual memory breaks RAM into small **pages** that can be stored anywhere, PagedAttention breaks the KV-cache into small pages that can be scattered across GPU memory.



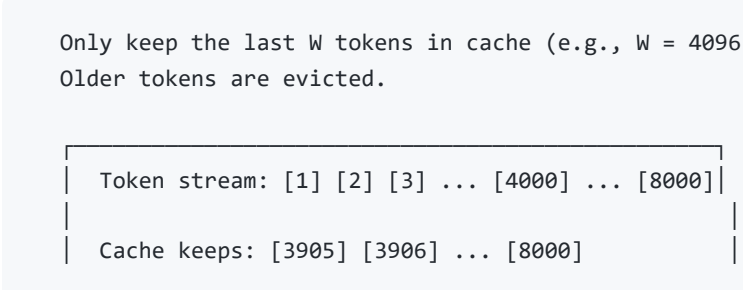
Result: 2–4× more concurrent requests on the same hardware, near-zero memory waste.

Used by: vLLM, TensorRT-LLM, SGLang — most production LLM serving stacks. If you're deploying an LLM to serve users, you're almost certainly using PagedAttention under the hood.

KV-Cache Compression

As context windows grow to 100K–1M tokens, the KV-cache becomes the dominant memory consumer. Several techniques compress it:

Sliding Window Attention:



```
│  Evicted:     [1] [2] ... [3904]                  │
└──────────────────────────────────────────────────┘

✓ Simple, constant memory
✗ Loses long-range context entirely
Used by: Mistral (combined with full attention in alternating layers)
```

H2O (Heavy Hitter Oracle):

```
Keep tokens that consistently get HIGH attention scores.
Evict tokens that nobody pays attention to.

Intuition: In most text, a few tokens are "anchor" tokens that
everything else attends to (e.g., the subject of the paragraph,
key nouns). Keep those, evict filler tokens.

✓ Retains important context
✓ Adaptive — different texts keep different tokens
✗ Slightly more complex than sliding window
```

StreamingLLM:

```
Keep the first few "attention sink" tokens + recent window.

┌──────────────────────────────────────────────────┐
│  [SINK₁][SINK₂][SINK₃][SINK₄] ... [recent W]  │
│   ↑ first 4 tokens          ↑ last W tokens     │
│   (attention sinks)         (sliding window)    │
└──────────────────────────────────────────────────┘

Discovery: The first few tokens in ANY sequence absorb
disproportionate attention, regardless of content. Removing
them breaks generation. Keeping them + recent window enables
infinite-length generation with constant memory.

✓ Enables infinite-length generation
✓ Constant memory usage
✗ Still loses middle context
```

Evaluation & Scaling

Perplexity

Intuition: "On average, how many words is the model choosing between at each step?"

A perplexity of 10 means the model is, on average, as uncertain as if it were choosing between 10 equally likely options at every token. Lower is better — it means the model is more confident

and usually more accurate.

```
The formula:

  PPL = e^(average cross-entropy loss)

  This is just the exponential of the average loss we computed earlier.
  It converts the abstract loss number into something interpretable:
  "equivalent number of choices."

Examples:
  GPT-5 on English text   ≈ 5–10 PPL    (choosing between ~5–10 options)
  Small model on English   ≈ 20–50 PPL   (much more uncertain)
  Random model (50K vocab) ≈ 50,000 PPL  (no idea, guessing uniformly)
  Perfect model            = 1 PPL       (knows every token in advance)
```

Important caveats:

- Perplexity is only comparable when using the **same tokenizer and dataset**. A model with a 32K vocabulary and one with a 100K vocabulary can't be directly compared.
- Low perplexity ≠ "good at tasks." A model can predict text well but still fail at reasoning. Perplexity measures language modeling, not intelligence.

Key Benchmarks

```
┌──────────────┬──────────────────────────────┬──────────────────────────┐
│ Benchmark    │ What It Tests                │ Example Question         │
├──────────────┼──────────────────────────────┼──────────────────────────┤
│ MMLU         │ 57-subject knowledge         │ "The Krebs cycle..."     │
│              │ (history, law, physics, etc.)│                          │
├──────────────┼──────────────────────────────┼──────────────────────────┤
│ HumanEval    │ Code generation (Python)     │ Write fibonacci()        │
│              │ functional correctness       │                          │
├──────────────┼──────────────────────────────┼──────────────────────────┤
│ GSM8K        │ Grade-school math reasoning  │ "If Alice has 3          │
│              │ (multi-step word problems)   │  apples and buys 5..."   │
├──────────────┼──────────────────────────────┼──────────────────────────┤
│ ARC          │ Science reasoning            │ "What causes tides?"     │
│              │ (grade-school level)         │                          │
├──────────────┼──────────────────────────────┼──────────────────────────┤
│ GPQA         │ PhD-level science questions  │ Quantum mechanics Q      │
│              │ (expert-validated)           │                          │
├──────────────┼──────────────────────────────┼──────────────────────────┤
│ SWE-bench    │ Real GitHub bug fixing       │ Fix this actual bug      │
│              │ (end-to-end software eng.)   │ in a real repository     │
├──────────────┼──────────────────────────────┼──────────────────────────┤
│ MATH         │ Competition-level math       │ Prove / solve a          │
```

```
│              │ (AMC, AIME difficulty)        │ competition problem    │
└──────────────┴──────────────────────────────┴────────────────────────┘
```

Caveat — benchmark contamination: A real concern. Models may have seen benchmark questions during pre-training (they're on the internet). This inflates scores. Always look at **held-out** benchmarks like SWE-bench (generated from real GitHub issues after training cutoff dates) for more reliable evaluation.

Chinchilla Scaling Law

Key insight (DeepMind, 2022): For a given compute budget, you should train a **smaller model on more data**, not a bigger model on less data.

```
┌──────────────────────────────────────────────────────────────────┐
│                    CHINCHILLA SCALING LAW                        │
│                                                                  │
│  The optimal ratio: ~20 tokens per parameter                     │
│                                                                  │
│  ┌──────────────────┬──────────────┬──────────────────────────┐  │
│  │ Model Size       │ Optimal Data │ Actual Training Data     │  │
│  ├──────────────────┼──────────────┼──────────────────────────┤  │
│  │ 7B params        │ 140B tokens  │ LLaMA-2: 2T tokens       │  │
│  │                  │              │ (14× "optimal"!)         │  │
│  │ 70B params       │ 1.4T tokens  │ LLaMA-2: 2T tokens       │  │
│  │                  │              │ (1.4× "optimal")         │  │
│  │ 175B params      │ 3.5T tokens  │ GPT-3: 300B tokens       │  │
│  │                  │              │ (0.09× — undertrained!)  │  │
│  └──────────────────┴──────────────┴──────────────────────────┘  │
│                                                                  │
│  The twist: "Chinchilla-optimal" minimizes TRAINING cost.        │
│  But for INFERENCE cost, over-training smaller models is better. │
│                                                                  │
│  Why? You pay for training ONCE, but inference MILLIONS of times.│
│  A 7B model trained on 2T tokens (14× over-trained) is:          │
│   - More expensive to train than "optimal"                       │
│   - But MUCH cheaper to serve (7B vs 70B per inference call)     │
│   - And nearly as capable as the "optimal" 70B trained on 1.4T   │
│                                                                  │
│  Modern strategy: train small models on as much data as possible.│
│  LLaMA 4, Mistral, Phi-3 all follow this over-training approach. │
└──────────────────────────────────────────────────────────────────┘
```

Inference-Time Compute Scaling

The new frontier: Instead of making models bigger (more parameters), make them **think longer** at inference time. Spend more compute per question, not per training run.



The key takeaway: The future of AI performance is not just "bigger models" — it's smarter inference. Agents, chain-of-thought, and search-based decoding are all ways to trade inference compute for better answers. This is why understanding agent architectures (Days 3–7 of this curriculum) is so important.

← Back to Day 1: LLM Foundations

Next → Day 2: Advanced Prompt Engineering